Leonor Rebola
Estudante nº 53663
Adriana Abreu
Estudante nº 53672
Covilhã, Abril de 2026

Lab06

O objetivo deste trabalho é colocar em prática as três camadas de proteção essenciais, que são elas a filtragem, a validação e a autenticação.

Filtragem

Foram utilizadas duas abordagens: a biblioteca **Bleach** (remoção de tags HTML) e a biblioteca **re** (permitindo apenas caracteres alfanuméricos e sinais controlados). Os inputs de teste incluíram tags `<script>`, injeções SQL, emojis e caracteres unicode.

```
import bleach
import re

def sanitizar_html(texto):
    return bleach.clean(texto, tags=[], attributes={}, strip=True
        )

def sanitizar_regex(texto, permitidos=r"[^a-zA-Z0-9 _.@\-]"):
    return re.sub(permitidos, "", texto)

inputs = [
    "<script>alert('XSS')</script>",
    "<b>Ola</b>, chamo-me Joao!",
    "email@exemplo.com; DROP TABLE users;--",
    "Texto normal com emojis e unicode: cafe",
    "<img src=x onerror=alert(1)>",
]

for inp in inputs:
    print(f"BLEACH : {sanitizar_html(inp)}")
    print(f"REGEX  : {sanitizar_regex(inp)}")
```

Input	**Bleach**	**Regex**
`<script>alert('XSS')</script>`	`alert('XSS')`	`scriptalertXSSscript`
`<b>Olá</b>, chamo-me João!`	`Olá, chamo-me João!`	`bOlb chamo-me Joo`
`email@ex.com; DROP TABLE users;--`	`inalterado`	`email@ex.com DROP TABLE users--`
`Texto com émojis e café`	`inalterado`	`Texto com mojis e caf`
`<img src=x onerror=alert(1)>`	`(vazio)`	`img srcx onerroralert1`

Tabela 1: Comparação entre Bleach e Regex para sanitização de inputs

O Bleach revelou-se a melhor solução para filtragem de HTML/XSS, removendo tags maliciosas e preservando o texto legível, eliminando também completamente o `<img onerror=...>`. O Regex foi mais destrutivo, removendo acentos, emojis e caracteres unicode legítimos, tornando-se inadequado para texto geral. Nenhuma das abordagens protege contra injeção SQL, sendo que essa proteção deve ser feita noutras camadas, como *prepared statements* ou ORMs.

Validação

Foi definido um esquema com a biblioteca **Pydantic**, com os campos `nome` (string não vazia), `idade` (inteiro entre 0 e 150), `email` (formato válido) e `data_nascimento`. Foram ainda criados validadores customizados com `@field_validator` para impor regras semânticas.

```
from pydantic import BaseModel, EmailStr, field_validator,
   ValidationError
from typing import Optional
from datetime import date

class Utilizador(BaseModel):
    nome: str
    idade: int
    email: EmailStr
    data_nascimento: Optional[date] = None

    @field_validator("nome")
    @classmethod
    def nome_nao_vazio(cls, v):
        if not v.strip():
            raise ValueError("O nome nao pode estar vazio.")
        return v.strip()

    @field_validator("idade")
    @classmethod
    def idade_valida(cls, v):
        if v < 0:
            raise ValueError("A idade nao pode ser negativa.")
        if v > 150:
            raise ValueError("A idade nao e plausivel.")
        return v
```

O Pydantic validou corretamente todos os casos, rejeitando inputs inválidos com mensagens claras e específicas. Todos os inputs válidos foram aceites e os inválidos rejeitados sem exceção, incluindo campos obrigatórios em falta, tipos incorretos e valores fora de intervalo.

Input	Resultado
{"nome": "Ana Silva", "idade": 25, "email": "ana@exemplo.com"}	Válido
{"nome": "João", "idade": 0, "email": "joao@ubi.pt", "data_nascimento": "2000-05-10"}	Válido
{"nome": , "idade": 25, "email": "ana@exemplo.com"}	nome: não pode estar vazio
{"nome": "Carlos", "idade": -5, "email": "carlos@ubi.pt"}	idade: não pode ser negativa
{"nome": "Maria", "idade": 30, "email": "emailinvalido"}	email: sem @-sign
{"nome": "Pedro", "idade": 200, "email": "pedro@ubi.pt"}	idade: não é plausível
{"idade": 22, "email": "sem_nome@ubi.pt"}	nome: Field required

Tabela 2: Resultados da validação de payload com Pydantic

Autenticação

Foram implementadas três técnicas com crescente nível de garantia: hash **SHA-256**, **HMAC** com chave secreta partilhada, e **assinatura digital RSA** com par de chaves pública/privada.

```
import hashlib, hmac
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa,
   padding
from cryptography.exceptions import InvalidSignature

# SHA-256
conteudo_original = b"Este e o meu contrato importante."
conteudo_alterado = b"Este e o meu contrato importanteX"
hash_original = hashlib.sha256(conteudo_original).hexdigest()
hash_alterado  = hashlib.sha256(conteudo_alterado).hexdigest()

# HMAC
chave_secreta = b"chave-super-secreta"
def gerar_hmac(chave, mensagem):
    return hmac.new(chave, mensagem, hashlib.sha256).hexdigest()

# RSA
chave_privada = rsa.generate_private_key(public_exponent=65537,
   key_size=2048)
chave_publica = chave_privada.public_key()
assinatura = chave_privada.sign(
    conteudo_original, padding.PKCS1v15(), hashes.SHA256()
```

```
)
```

A alteração de um único byte produziu hashes e HMACs completamente distinto. O HMAC acrescentou autenticidade à integridade, em que uma chave errada gera um valor diferente, impedindo falsificação. A assinatura RSA é a solução mais robusta, garantindo integridade e autenticidade sem partilhar a chave secreta, sendo a base de protocolos como TLS e JWT. O uso de `hmac.compare_digest` em vez de `==` é igualmente relevante, pois é resistente a *timing attacks*.

Técnica	**Integridade**	**Autenticidade**	**Observações**
SHA-256	Sim	Não	Qualquer um pode recalcular
HMAC	Sim	Sim (chave partilhada)	Precisa de chave secreta comum
RSA	Sim	Sim (chave pública/privada)	Mais robusto, não partilha chave

Tabela 3: Comparação entre técnicas de autenticação

Conclusão

As três técnicas estudadas formam camadas complementares de proteção. A **filtragem** impede que dados maliciosos entrem no sistema. A **validação** garante que os dados respeitam o formato e as regras de negócio. A **autenticação** assegura que os dados não foram alterados e provêm de uma fonte legítima. A aplicação conjunta destas práticas é essencial em qualquer pipeline de dados ou arquitetura de aplicações modernas, reduzindo significativamente a superfície de ataque e os erros de integridade.